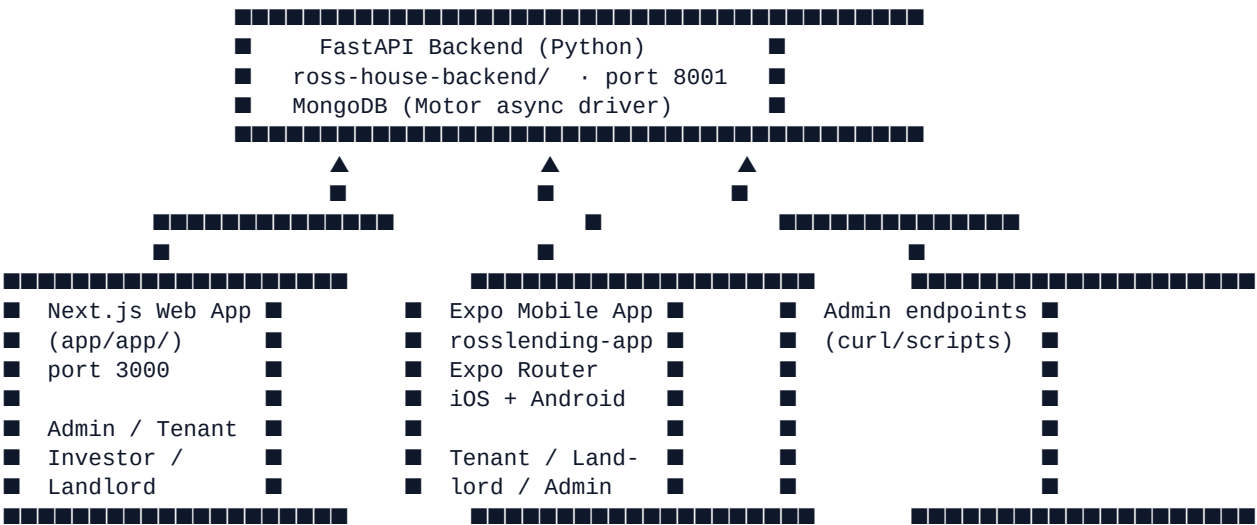


Ross House Rentals — Arquitectura Completa del Proyecto

- > Documento de referencia para compartir con otro agente (por ejemplo, el agente que está construyendo **Ross Travel Agency**) para que entienda el patrón de arquitectura que se usa en esta plataforma y replique las mejores prácticas.
- >
- > Última actualización: Junio 2026
- > Propietario: Yoandy Ross
- > Stack: **FastAPI + MongoDB + Next.js (App Router) + Expo (React Native)**

1. Visión General

Ross House Rentals es una **plataforma full-stack multi-rol** para administración de propiedades de alquiler. Tiene **3 aplicaciones independientes** que consumen el mismo backend:



Roles soportados

Rol	Descripción
admin	Yoandy + staff. CRUD total, dashboard financiero, moderación marketplace
tenant	Inquilinos. Pagan renta, ven contratos, suben recibos, OCR de utilities
investor	Inversores en sindicaciones inmobiliarias

Rol	Descripción
landlord	Propietarios externos que listan propiedades en el Marketplace
guest	Usuarios públicos / leads del Marketplace

2. Backend — ross-house-backend/

2.1 Stack y configuración

- **Framework:** FastAPI (Python 3.11)
- **DB:** MongoDB Atlas (driver **motor** asyncio)
- **Auth:** JWT (HS256) — **python-jose**, **bcrypt**
- **PDFs:** **reportlab** / **weasyprint** (contratos, recibos, reportes)
- **Deploy:** Railway/Render (**Procfile**, **nixpacks.toml**, **runtime.txt**)
- **Entrypoint:** **server.py** → registra todos los routers de **rental/**

2.2 Estructura de carpetas

```

ross-house-backend/
■■■ server.py                # FastAPI app + router registration + CORS
■■■ requirements.txt
■■■ Procfile
■■■ nixpacks.toml
■■■ rental/                  # ★ Módulo principal: routers de dominio
■   ■■■ auth_router.py      # Login, registro, JWT, password reset
■   ■■■ tenant_router.py    # Endpoints del inquilino
■   ■■■ properties_router.py # CRUD de propiedades
■   ■■■ owner_router.py     # CRM propietarios externos
■   ■■■ contracts_router.py  # Contratos de arriendo (rental_contracts)
■   ■■■ finances_router.py   # Dashboard financiero admin
■   ■■■ investments_router.py # Inversiones / Real Estate Syndication
■   ■■■ syndication_router.py # Deals de sindicación con cap table
■   ■■■ credit_builder_router.py # Credit reporting (MOCKED Experian)
■   ■■■ vault_router.py      # Bóveda PIN-protegida de documentos
■   ■■■ consent_forms_router.py # Formularios de consentimiento firmados
■   ■■■ signatures_router.py  # E-signature legalmente vinculante
■   ■■■ communications_router.py # SMS / Email / Push notifications
■   ■■■ chat_router.py       # Chat tenant ↔ admin
■   ■■■ faq_router.py        # FAQ AI con embeddings
■   ■■■ ai_brain_router.py    # AI assistant (GPT-4o via Emergent LLM)
■   ■■■ utility_ocr_router.py  # OCR de facturas (GPT-4o Vision)
■   ■■■ utility_billing_router.py # Reparto de utilities (kWh/agua/gas)
■   ■■■ utility_payments_router.py
■   ■■■ tenant_utilities_router.py
■   ■■■ xcel_energy_router.py  # Green Button API (SAML SSO, BLOQUEADO)
■   ■■■ mashvisor_routes.py   # Comps inmobiliarios (Mashvisor API)
■   ■■■ reports_router.py     # PDFs de reportes admin
■   ■■■ legal_router.py      # ToS / Privacy / E-sign disclosures

```

```

■ ■■■ autopay_cron.py          # Cron de cobro automático mensual
■ ■■■ rent_payment_cron.py     # Aviso/recordatorios renta
■ ■■■ property_sync_cron.py    # Sincroniza datos de propiedades
■ ■■■ tenant_invoices_router.py # Invoices CRUD admin
■ ■■■ stripe_pkg/              # ★ Sub-paquete Stripe Connect
■ ■ ■■■ tenant_payments_router.py # PaymentIntents tenant
■ ■ ■■■ connect_router.py      # Stripe Connect (landlords)
■ ■ ■■■ payment_methods_router.py # Cards / ACH / bank
■ ■ ■■■ autopay_router.py      # Suscripciones autopay
■ ■ ■■■ admin_config_router.py  # Config llaves Stripe
■ ■ ■■■ webhooks_router.py     # Webhooks de eventos Stripe
■ ■■■ shared.py                # DB connection, JWT decode, helpers
■■■ rental_pdf_service.py      # Generador de PDFs de leases
■■■ rental_storage_service.py  # S3/local storage helpers
■■■ push_notification_service.py
■■■ scripts/                   # Migraciones y seeds one-off

```

2.3 Convención de rutas

TODA ruta del backend está prefijada con **/api** (regla de Kubernetes ingress).

- Ejemplo: **GET /api/admin/owners**, **POST /api/tenant/payments**
- CORS: configurado para Next.js (puerto 3000) y Expo (puerto 8081/web)

2.4 Colecciones MongoDB principales

Colección	Descripción
users	Cuentas + roles (admin/tenant/investor/landlord)
properties	Propiedades internas de Ross House
owners	Propietarios externos (CRM)
rental_contracts	Contratos firmados (migrados de leases)
payments / tenant_payments	Pagos de renta vía Stripe
tenant_invoices	Invoices generadas para inquilinos
utility_bills	Facturas de servicios (OCR + manual)
utility_charges	Cargos prorrateados por tenant
marketplace_listings	Listados públicos de propiedades de landlords
landlord_commissions	Comisiones cobradas por leads
investments / syndications	Deals de inversión + cap table
credit_builder_enrollments	Tenants inscritos a credit reporting (MOCKED)
vault_documents	Docs sensibles con PIN
signatures	Firmas legales con IP/timestamp/hash
notifications	Inbox de notifs por usuario

2.5 Integraciones de terceros

Servicio	Uso	Key Source
Stripe Connect	Pagos renta + payouts landlords	User-provided
SendGrid	Emails transaccionales	User-provided
Twilio	SMS / OTP	User-provided
OpenAI GPT-4o	OCR Vision + chatbot	Emergent LLM Key
Mashvisor	Comps inmobiliarios	User-provided
Xcel Energy	Green Button kWh (SAML SSO)	BLOQUEADO
Experian	Credit reporting	MOCKED (sin keys)

3. Web App — app/app/ (Next.js 14 App Router)

3.1 Stack

- **Framework:** Next.js 14 (App Router, Server Components)
- **UI:** Tailwind CSS + shadcn/ui + lucide-react
- **State:** React Hooks + **fetch** directo al backend
- **i18n:** **app/i18n/** (es + en) — usuario es bilingüe pero por defecto **español**
- **Deploy:** Vercel (gatillado por "Save to GitHub" en Emergent UI)
- **Layout raíz:** **app/layout.tsx** (font, providers, theme)

3.2 Estructura de carpetas (rutas principales)

```
app/  
  layout.tsx           # Root layout, font, providers  
  page.tsx             # Landing pública  
  globals.css          # Tailwind + fixes (ej. iOS select dropdown)  
  i18n/                # JSON traducciones  
  sections/            # Componentes de la landing  
  components/          # UI compartido (botones, modals, tablas)  
    
  admin/               # ★ PORTAL ADMIN (yoandyross@gmail.com)  
    layout.tsx         # Sidebar + auth guard de rol admin  
    page.tsx + page-client.tsx # Dashboard financiero  
    propiedades/       # CRUD propiedades  
    propietarios/      # CRM owners (creado en esta sesión)  
    inquilinos/        # CRUD tenants  
    inversionistas/    # CRUD investors  
    contratos/         # Leases/contracts admin  
    pagos/             # Reconciliación de pagos  
    facturas-ocr/      # Upload + OCR GPT-4o Vision  
    gastos/            # Gastos operativos  
    mantenimiento/    # Tickets de mantenimiento  
    inspecciones/      # Inspecciones con fotos  
    energia/           # Dashboard kWh Xcel
```

```

■ ■■■ alineacion-utilities/      # Reparto utility bills
■ ■■■ autopagos/                # Configuración cron autopay
■ ■■■ metodos-pago/             # Stripe keys + métodos
■ ■■■ marketplace/             # Moderación de listings + fotos + CRUD
■ ■ ■■■ comisiones/            # Comisiones cobradas
■ ■■■ credit-builder/          # Panel Credit Builder (creado en esta sesión)
■ ■■■ baul/                    # Bóveda PIN
■ ■■■ syndication/            # Deals de sindicación + [id] detail + new
■ ■■■ inversiones/            # Performance financiero por inversionista
■ ■■■ rendimiento/            # KPI cards
■ ■■■ reportes/                # Generador de PDFs
■ ■■■ calendario/             # Eventos + recordatorios
■ ■■■ mensajes/                # Chat inbox admin
■ ■■■ aplicaciones/           # Aplicaciones de inquilinato
■ ■■■ configuracion/          # Settings generales
■
■■■ tenant/                    # ★ PORTAL INQUILINO
■ ■■■ pagar/                  # Pagar renta
■ ■■■ contratos/              # Ver/firmar contrato
■ ■■■ recibos/                # Subir recibos
■ ■■■ utilities/              # Reparto de utilities + OCR
■ ■■■ perfil/
■
■■■ inversor/                  # Portal investor (read-only)
■ ■■■ ... (dashboards, posiciones, distribuciones)
■
■■■ invest/                    # Páginas públicas de cada deal de sindicación
■
■■■ landlord/                  # ★ PORTAL LANDLORD
■ ■■■ listings/               # CRUD de sus listings marketplace
■ ■■■ onboarding/             # Stripe Connect onboarding
■ ■■■ comisiones/
■ ■■■ perfil/
■
■■■ privacy-policy/           # Páginas legales públicas

```

3.3 Patrones de UX

- **iOS select fix:** `globals.css` con `color: black !important` para `<select>`
- **Toast/Modals:** componentes propios bajo `app/components/`
- **Auth guard:** verificación de rol al inicio del layout de cada portal
- **API client:** `fetch` directo con `NEXT_PUBLIC_BACKEND_URL` desde `.env`

4. Mobile App — `rosslending-app/` (Expo + Expo Router)

4.1 Stack

- **Framework:** Expo SDK 51+ con **Expo Router (file-based)**
- **UI:** React Native core + custom theme en `src/constants/theme.ts`
- **State:** React Context (`AuthContext`) + AsyncStorage


```

■■■ utils/
    ■■■ api.ts                # Wrapper fetch con auth header
    ■■■ formatters.ts        # currency, dates
    ■■■ pushNotifications.ts  # Expo + FCM token registration

```

4.3 Patrones clave

- **Variables de entorno:** `EXPO_PUBLIC_BACKEND_URL` en `.env`
- **Auth flow:** JWT en `AsyncStorage`, `AuthContext` decodifica rol y enruta
- **Plataforma dual:** archivos `.web.tsx` para overrides web (Stripe.js vs native SDK)
- **Permisos contextuales:** cámara solo cuando el usuario presiona "Scan Bill"
- **Builds:** SOLO via "Publish" en Emergent UI, jamás `eas build` directo

5. Convenciones Globales (replicables a Ross Travel Agency)

5.1 Reglas de oro

1. **Backend prefix** `/api` para que Kubernetes ingress redirija al puerto correcto.
2. **NUNCA hardcodear URLs/puertos.** Usar variables de entorno:
 - Frontend Next.js: `NEXT_PUBLIC_BACKEND_URL`
 - Expo: `EXPO_PUBLIC_BACKEND_URL`
 - Backend: `MONGO_URL` desde `.env`
3. **Roles** desde un solo enum, validado en cada router con dependencia `Depends(get_current_user)`.
4. **i18n primero (es por defecto, en secundario)** — no hardcodear strings en JSX.
5. **PDFs** generados server-side con `reportlab` y devueltos como `StreamingResponse`.
6. **Cron jobs** como módulos `*_cron.py` ejecutados via APScheduler dentro de FastAPI.
7. **Mocked integrations:** marcar claramente con comentario `# MOCKED — pending API keys` y un flag en `.env`.

5.2 Deployments

App	Cómo se despliega
FastAPI backend	<code>git push</code> → Railway/Render auto-deploy
Next.js Web	Botón "Save to GitHub" en Emergent UI → Vercel auto-deploy
Expo Mobile	Botón "Publish" en Emergent UI → EAS Build (NUNCA manual)

5.3 Seguridad / Compliance

- Passwords: bcrypt cost 12
- JWT: HS256, expiración 24h, refresh 30d
- Bóveda PIN: PBKDF2 + SHA-256 sobre PIN del usuario
- E-sign: hash SHA-256 del PDF firmado + IP + UA + timestamp guardado
- Stripe: webhooks verificados con **STRIPE_WEBHOOK_SECRET**
- Permisos móvil: contextual (regla **<handle_permissions_contract>**)

5.4 Estructura recomendada para CLONAR este patrón

Si quieres aplicar la misma arquitectura a **Ross Travel Agency** (u otro proyecto):

```
ross-travel/
├── ross-travel-backend/      # FastAPI con dominio "travel" en /api/travel/*
│   ├── server.py
│   └── travel/              # equivalente a "rental/"
│       ├── auth_router.py
│       ├── bookings_router.py
│       ├── flights_router.py
│       ├── hotels_router.py
│       ├── payments_router.py
│       ├── stripe_pkg/
│       └── requirements.txt
├── app/app/                 # Next.js: admin/agent/customer portals
│   ├── admin/               # Travel agent backoffice
│   ├── customer/            # Cliente final
│   └── partner/             # Hotel/aerolínea partner
└── ross-travel-app/         # Expo Router con (tabs)/(auth)
    ├── app/(tabs)/index.tsx # Búsqueda
    ├── app/(tabs)/bookings.tsx # Mis reservas
    └── src/contexts/AuthContext.tsx
```

6. Cuentas de prueba (sandbox)

> Archivo fuente: **/app/memory/test_credentials.md**

Rol	Email	Password
Admin	yoandyross@gmail.com	admin123
Tenant	anaelish88@gmail.com	Admin.123
Landlord	demo.landlord@rosshouse.test	Demo123!
Guest	demo.tenant@rosshouse.test	Demo123!

7. Estado actual (Junio 2026)

- ■ Marketplace público + Landlord onboarding + Comisiones
- ■ CRM Propietarios
- ■ Credit Builder Admin (Experian MOCKED)
- ■ E-sign de contratos + PDF preview móvil
- ■ OCR Utility bills (GPT-4o Vision)
- ■ Autopay Cron + Vault PIN
- ■ April Tax API integration (próxima tarea)
- ■ Xcel Energy Green Button (bloqueado por vendor)

FIN DEL DOCUMENTO — Compartir este archivo (Markdown) al otro agente para que tenga el blueprint completo de cómo está estructurado Ross House Rentals y replique el patrón en cualquier vertical (travel, lending, tax, etc.).